

Serving and Deploying Models

Turn a fitted model into a validated network service and run it safely in the cloud.



WHY THIS MATTERS

Start from what you already know

Python functions, dictionaries, validation, fitted pipelines, and inference

- 01 Explain HTTP and JSON in plain language.
- 02 Use FastAPI contracts in a small worked example.
- 03 Complete the guided lab and verify AWS operations.

CORE CONCEPT

The vocabulary of this topic

Turn a fitted model into a validated network service and run it safely in the cloud.

HTTP and JSON: HTTP defines request and response exchange; JSON represents structured values in the message body.

FastAPI contracts: A FastAPI contract declares an endpoint's method, typed request, response, and error behavior.

Docker containers: A Docker container runs an application inside the filesystem and dependencies fixed by its image.

AWS operations: AWS operations cover scoped identity, networking, compute, logs, health, budgets, and controlled rollback.

FOUR CONNECTED IDEAS

How the concept works

01

**HTTP and
JSON**

02

**FastAPI
contracts**

03

**Docker
containers**

04

**AWS
operations**

Client → HTTP request → validated schema → model inference → JSON response; a container fixes the runtime environment.

CORE CONCEPT

Read every part of the mechanism

Client → HTTP request → validated schema → model inference → JSON response; a container fixes the runtime environment.

endpoint: A network address and HTTP method.

schema: Required fields and types.

container: A packaged application runtime.

WORKED EXAMPLE

Worked example by hand

Send age=30 and income=50000 to `/predict`; validate types; return prediction and model version.

01

Known values

02

Apply the mechanism

03

Interpret the result

RUN IT AND INSPECT THE RESULT

Map the concept to Python

```
@app.post("/predict")
def predict(request: PredictionRequest):
    value = model.predict([request.model_dump().values()])[0]
    return {"prediction": float(value), "model_version": "1"}
```

OUTPUT

```
{"prediction": 0.72, "model_version": "1"}
```

Each line corresponds to a concept already explained.

QUALITY GATE

Build the complete mental model

- ☒ **Validate request fields before inference and return structured 4xx errors for invalid client input.**

- ☒ **Package the same fitted preprocessing pipeline used during training to prevent serving skew.**

- ☒ **Use immutable container tags or digests, scoped IAM roles, and secrets outside source code.**

- ☒ **Deploy behind a health check with logs, budgets, resource limits, and documented rollback.**

FOUR CONNECTED IDEAS

Notebook readiness map

01

**HTTP and
JSON**

02

**FastAPI
contracts**

03

**Docker
containers**

04

**AWS
operations**

Use the concepts from this lesson to read and complete
<https://github.com/curiously/Al-Bootcamp/blob/master/README.md>.

RISK REVIEW

Common beginner mistakes

01 **Accepting malformed input and passing it to the model.**

02 **Opening cloud ports broadly or omitting cost limits.**

GUIDED LAB

Expose a model through FastAPI, containerize it, add health checks, and document a minimal free-tier EC2 deployment.

01

02

03

04

Predict

Run

Inspect

Explain

SESSION SYNTHESIS

Keep the system, not isolated definitions.

- 01 Explain HTTP and JSON in plain language.
- 02 Use FastAPI contracts in a small worked example.
- 03 Complete the guided lab and verify AWS operations.

NEXT EVIDENCE

A tested API, Docker image, deployment runbook, health endpoint, logs, budget alert, and rollback steps.